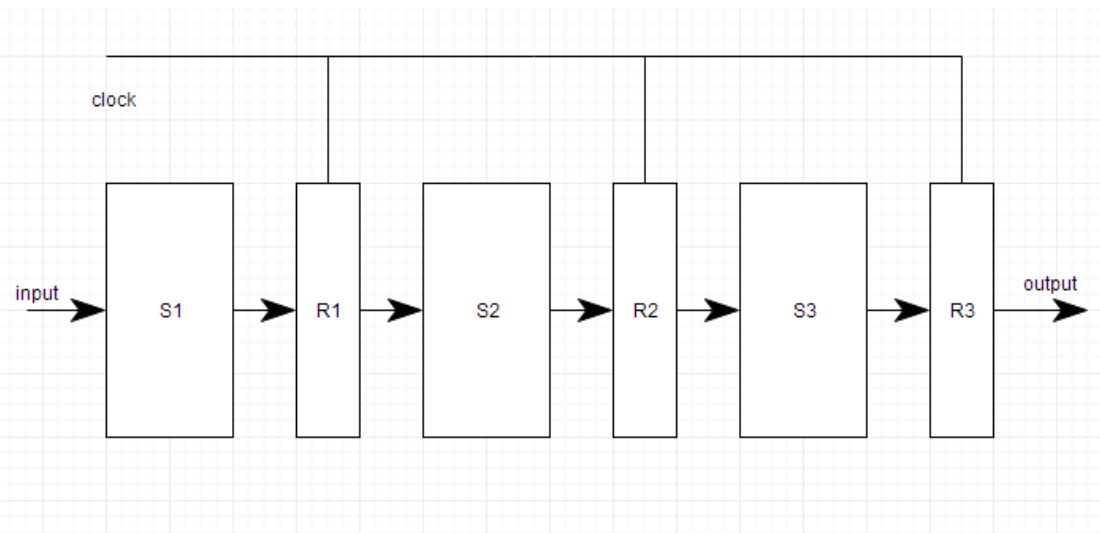


• Pipelining

To improve the performance of a CPU we have two options: 1) Improve the hardware by introducing faster circuits. 2) Arrange the hardware such that more than one operation can be performed at the same time. Since there is a limit on the speed of hardware and the cost of faster circuits is quite high, we have to adopt the 2nd option.

- **Pipelining** is a process of arrangement of hardware elements of the CPU such that its overall performance is increased.
- Simultaneous execution of more than one instruction takes place in a pipelined processor.
- Pipelining is the process of accumulating instruction from the processor through a pipeline. It allows storing and executing instructions in an orderly process. It is also known as **pipeline processing**.
- Pipelining is a technique where multiple instructions are overlapped during execution.
- Pipeline is divided into stages and these stages are connected with one another to form a pipe like structure. Instructions enter from one end and exit from another end.
- **Pipelining increases the overall instruction throughput.**
- In pipeline system, each segment consists of an input register followed by a combinational circuit. The register is used to hold data and combinational circuit performs operations on it. The output of combinational circuit is applied to the input register of the next segment.



- **Performance of a pipelined processor**

Consider a 'k' segment pipeline with clock cycle time as 'Tp'. Let there be 'n' tasks to be completed in the pipelined processor. Now, the first instruction is going to take 'k' cycles to come out of the pipeline but the other 'n - 1' instructions will take only '1' cycle each, i.e, a total of 'n - 1' cycles. So, time taken to execute 'n' instructions in a pipelined processor:

$$ET_{\text{pipeline}} = k + n - 1 \text{ cycles} \\ = (k + n - 1) T_p$$

In the same case, for a non-pipelined processor, the execution time of 'n' instructions will be:

$$ET_{\text{non-pipeline}} = n * k * T_p$$

So, speedup (S) of the pipelined processor over the non-pipelined processor, when 'n' tasks are executed on the same processor is:

$$S = \text{Performance of pipelined processor} / \text{Performance of non-pipelined processor}$$

As the performance of a processor is inversely proportional to the execution time, we have,

$$S = ET_{\text{non-pipeline}} / ET_{\text{pipeline}} \\ \Rightarrow S = [n * k * T_p] / [(k + n - 1) * T_p] \\ S = [n * k] / [k + n - 1]$$

When the number of tasks 'n' is significantly larger than k, that is, $n \gg k$

$$S = n * k / n \\ S = k$$

where 'k' are the number of stages in the pipeline.

Also, **Efficiency** = Given speed up / Max speed up = S / S_{max} We know that $S_{\text{max}} = k$

So, **Efficiency** = S / k

Throughput = Number of instructions / Total time to complete the instructions

So, **Throughput** = $n / (k + n - 1) * T_p$

• Types of Pipeline

It is divided into 2 categories:

1. Arithmetic Pipeline
2. Instruction Pipeline

Arithmetic Pipeline

Arithmetic pipelines are usually found in most of the computers. They are used for floating point operations, multiplication of fixed point numbers etc. For example: The input to the Floating Point Adder pipeline is:

$$X = A * 2^a$$

$$Y = B * 2^b$$

Here A and B are mantissas (significant digit of floating point numbers), while **a** and **b** are exponents.

Instruction Pipeline

In this a stream of instructions can be executed by overlapping *fetch*, *decode* and *execute* phases of an instruction cycle. This type of technique is used to increase the throughput of the computer system.

An instruction pipeline reads instruction from the memory while previous instructions are being executed in other segments of the pipeline. Thus we can execute multiple instructions simultaneously. The pipeline will be more efficient if the instruction cycle is divided into segments of equal duration.



however, results in several areas of inefficiency with respect to energy consumption such as unnecessary register file accesses, checking for forwarding and hazards when they cannot occur, latching unused values between pipeline registers and repeatedly calculating invariant values such as branch target addresses.

Pipeline Conflicts

There are some factors that cause the pipeline to deviate its normal performance. Some of these factors are given below:

1. Timing Variations

All stages cannot take same amount of time. This problem generally occurs in instruction processing where different instructions have different operand requirements and thus different processing time.

2. Data Hazards

When several instructions are in partial execution, and if they reference same data then the problem arises. We must ensure that next instruction does not attempt to access data before the current instruction, because this will lead to incorrect results.

3. Branching

In order to fetch and execute the next instruction, we must know what that instruction is. If the present instruction is a conditional branch, and its result will lead us to the next instruction, then the next instruction may not be known until the current one is processed.

4. Interrupts

Interrupts set unwanted instruction into the instruction stream. Interrupts effect the execution of instruction.

5. Data Dependency

It arises when an instruction depends upon the result of a previous instruction but this result is not yet available.

Advantages of Pipelining

1. The cycle time of the processor is reduced.
 2. It increases the throughput of the system
 3. It makes the system reliable.
-

Disadvantages of Pipelining

1. The design of pipelined processor is complex and costly to manufacture.
2. The instruction latency is more.

- **Why Processors have to be synchronized in same clock?**



Every computer has a physical clock that counts oscillations of a crystal. This hardware clock is used by the computer's software clock to track the current time. However, the hardware clock is subject to *drift*-- the clock's frequency varies and the time becomes inaccurate. As a result, any two clocks are likely to be slightly different at any given time. Ganeriwal reports that motes can lose up to 40 microseconds every second. The difference between two clocks is called their *skew*.

Generally, timing is a challenging and important issue in building distributed systems. Consider a couple of examples:

- A distributed, real time auction where the system must know which of two bidders submitted their bid first.
- A TDMA protocol that requires coordination among sensor nodes.
- A MAC protocol that requires coordination among sensor nodes.
- An application that determines the angle of arrival of an acoustic signal by analyzing the times at which the signal reaches an array of sensors.

There are several methods for synchronizing physical clocks. *External synchronization* means that all computers in the system are synchronized with an external source of time (e.g., a UTC signal). *Internal synchronization* means that all computers in the system are synchronized with one another, but the time is not necessarily accurate with respect to UTC.

• Pipelining Hazards

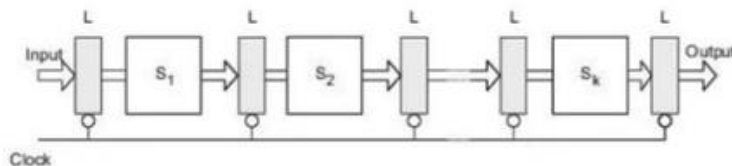
In a typical computer program besides simple instructions, there are branch instructions, interrupt operations, read and write instructions. Pipelining is not suitable for all kinds of instructions. When some instructions are executed in pipelining they can stall the pipeline or flush it totally. This type of problems caused during pipelining is called **Pipelining Hazards**.

➤ In most of the computer programs, the result from one instruction is used as an operand by the other instruction. When such instructions are executed in pipelining, break down occurs as the result of the first instruction is not available when instruction two starts collecting operands. So, instruction two must stall till instruction one is executed and the result is generated. This type of hazard is called Read -after-write pipelining hazard or **Data Hazard**.

➤ Execution of branch instructions also causes a pipelining hazard. Branch instructions while executed in pipelining effects the fetch stages of the next instructions. When the decision to execute one instruction is reliant on the result of another instruction, such as a conditional branch, which examines the condition's consequent value, a conditional hazard develops. It called **Branch Hazard** or **Control Hazard**.

❑ Two categories of pipelines:

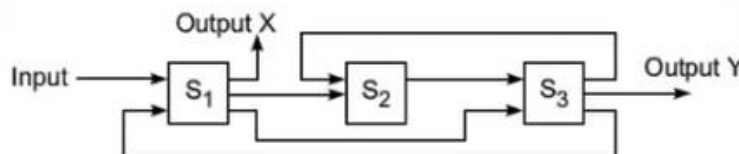
- ❑ **Linear pipeline:** It has **streamline (cascade)** connections only



Static Pipeline

Performing only one Function

- ❑ **Non-linear pipeline:** It has **feed-back connections and feed-forward connections** along with streamline connections



Dynamic Pipeline

Performing multiple (X and Y) Functions

- **Reservation Table :**

- The utilization pattern of successive stages in a pipeline is specified by **Reservation Table**. Rows represents **Stages** and columns represents **clock time units**.
- Total no. of clock units in a table is called **evaluation time** for a given function.
- A reservation table represents the flow of data through the pipeline for one complete evaluation of a given function. i.e. Each function evaluation is specified by one reservation table.
- Multiple reservation tables for different functions
 - A static pipeline or linear pipeline is specified by a single reservation table.
 - A dynamic pipeline or dynamic pipeline may be specified by more than one reservation table.

→ Time

	1	2	3	4	5	6	7	8
Stages S ₁	X					X		X
S ₂		X		X				
S ₃			X		X		X	

Reservation table for function X

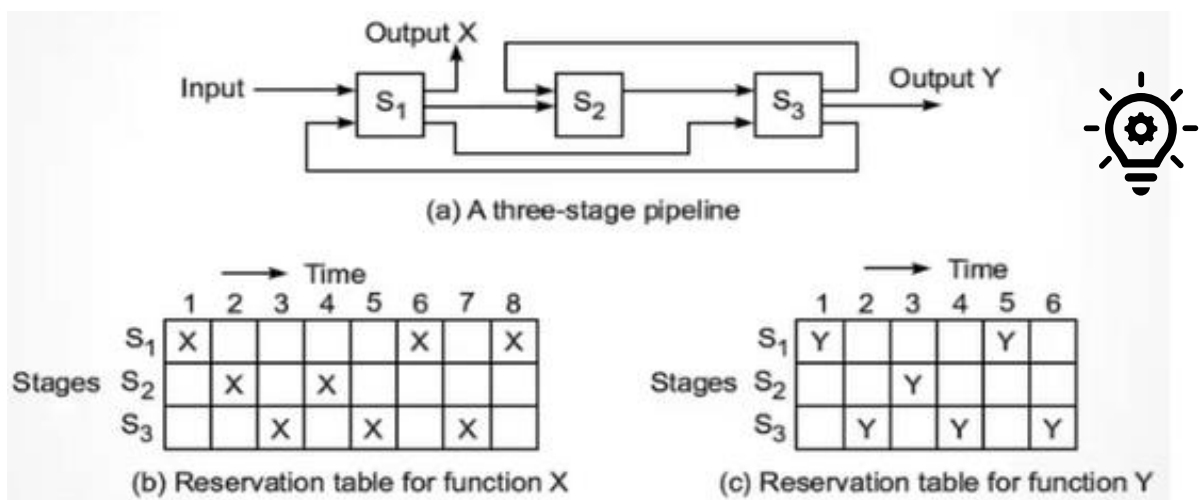
→ Time

	1	2	3	4	5	6
Stages S ₁	Y				Y	
S ₂			Y			
S ₃		Y		Y		Y

Reservation table for function Y

Total Evaluation time

- **Example**



Evaluation time of different functions may be different

Function X requires 8 clock cycles to evaluate.

Function Y requires 6 clock cycles to evaluate.

A Check Mark in a reservation table indicates that corresponding stage is used **at that time**.

Multiple check marks in a row indicates **repeated usage** of the same stage in different cycles.

Multiple check marks in a **column** indicates **simultaneous usage** of multiple stages in same cycle

Contiguous check marks in a row indicates **extended usage** of a stage over more than one cycle i.e. slower stage needing more time slots.

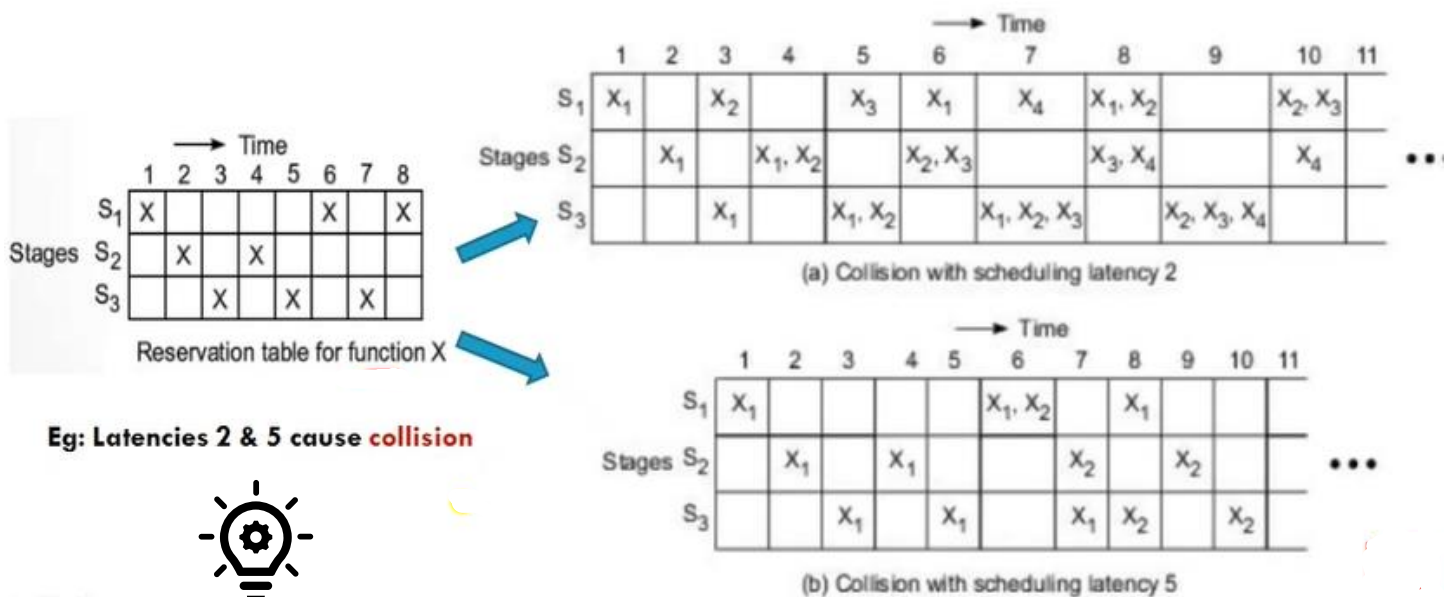
- **Latency:**

- An initiation refers to the start of single function evaluation
- The number of clock cycles between two initiations of a pipeline is the latency between them.
- Latency values must be positive integers.

- **Collision:**

- An attempt by two or more initiations to use the same pipeline stage at the same time is called collision.
- Collision implies Resource Conflicts between two initiations.
- Collisions must be avoided.

"Some latencies cause collisions and some not."



- **Forbidden Latencies:**

Latencies that **cause collisions** are called forbidden latencies.

To detect a forbidden latency, one needs simply to check the distance between any 2 checkmarks in the same row of the reservation table.

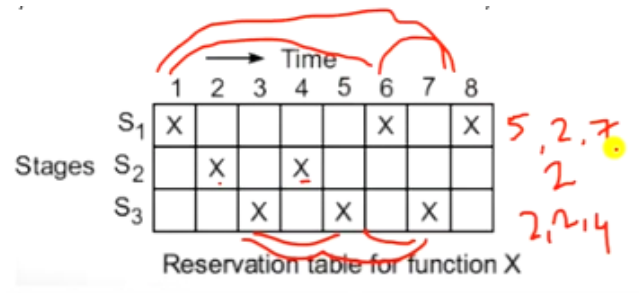
Example: Consider the reservation table

S1: {(6-1), (8-1), (8-6)} i.e. {5, 7, 2}

S2: {(4-2)}

S3: {(5-3), (7-3), (7-5)} i.e. {2, 4, 2}

Forbidden latencies = {2, 4, 5, 7}



- **Permissible Latencies:**

Latencies that does not cause collisions are permissible latencies.

(for the given table) Permissible latencies = {1, 3, 6, 8}

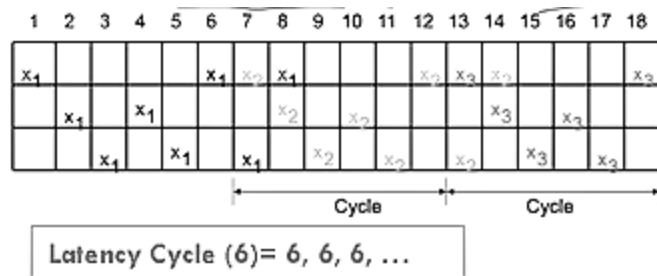
latency sequence:

A Latency sequence is a sequence of permissible latencies between successive task initiations

Eg: Latency sequence = {1, 3, 6, 8}

Latency Cycle:

A latency sequence that repeats the same subsequence (cycle) indefinitely.



- **Average Latency**

The Average Latency of a latency cycle is the sum of all latencies divided by the number of latencies along the cycle.

The Latency cycle (1, 8) has an average Latency of $(1 + 8) / 2 = 4.5$

- **Collision Free Scheduling**

To avoid collisions and to achieve high throughput, all the tasks waiting for initiation should be properly scheduled.

When scheduling events in a nonlinear pipeline, the main objective is:

- to obtain the shortest average latency between initiations without causing collisions

Following are the steps to obtain this:

1. Collision Vector
2. State Diagram
3. Greedy Cycles
4. MAL (Minimum average Latency)

- **Collision Vector:**



Collision vector is an m-bit binary vector $C = (C_m C_{m-1} \dots C_2 C_1)$ that can be obtained from the combined set of **permissible and forbidden latencies**.

(where $m \leq n-1$ in a n column reservation table)

$C_i = 1$ if latency i causes a collision i.e. for forbidden latencies
and $C_i = 0$ for permissible latencies.

Collision Vector = (1011010) (for the given reservation table : right to left)

Exercise :



		Time →							
		1	2	3	4	5	6	7	8
Stages	S ₁	X					X		X
	S ₂		X		X				
	S ₃			X		X		X	

Reservation table for function X

What are the forbidden latencies?

Draw the state transition diagram.

List all the simple cycles and greedy cycles.

Determine the minimal average latency (MAL).

S1: $\{(6-1), (8-6), (8-1)\}$ i.e. **{5, 2, 7}**

S2: $\{(4-2)\}$ i.e. **{2}**

S3: $\{(5-3), (7-5), (7-3)\}$ i.e. **{2, 2, 4}**

Forbidden latencies = {2, 4, 5, 7}

> It causes collision

Permissible Latency = {1, 3, 6, 8+}

> It does not causes collision

Maximum forbidden latency (m) = 7,

■ So the **collision vector** is of 7 bits

Collision Vector = {C₇ C₆ C₅ C₄ C₃ C₂ C₁}
1 0 1 1 0 1 0

- Draw State Transition diagram

A state diagram is constructed from Collision Vector. It specifies the permissible state transitions among successive initiations based on the collision vector.

How to draw state transition diagram?

The Permissible Latency = {1, 3, 6, 8+} so we have to find the next states of collision vector for the transitions (1 3 6 8+)

Method: To find the collision vector of the next state the collision vector of the present state is shifted

right for each transitions (1 . 3. 6. 8⁺) The shifted collision vector of present state is bitwise-OR ed with Initial Collision Vector (ICV).

Step 1: Calculating next state with Present State (PS) = 1011010

Permissible latencies (or transitions) = 1, 3, 6, 8⁺

Initial Collision Vector = 1011010

With latency 1:

1011010 ICV

0101101 After Right Shift PS by 1

1111111

so, the new-state-1111111 with latency 1

With latency 3:

1011010 ICV

0001011 After Right Shift PS by 3

1011011

With latency 6:

1011010

0000001

1011011

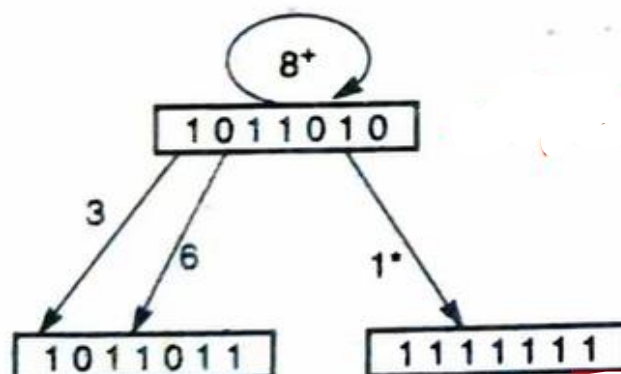
With latency 8 :

1011010

0000000

1011010

So the latency 8 returns to original ICV (101 1010)



Step 2: Calculating Next state with Present State (PS) = 1011011

Permissible latencies (or transitions) = 3, 6, 8+.

With Latency 3:

1011010

0001 01 1 After right shifting PS by 3

1011011 Next State

With latency 6:

1011010

0000001

1011011 Next State

So the latency 3 & 6 returns to same state.

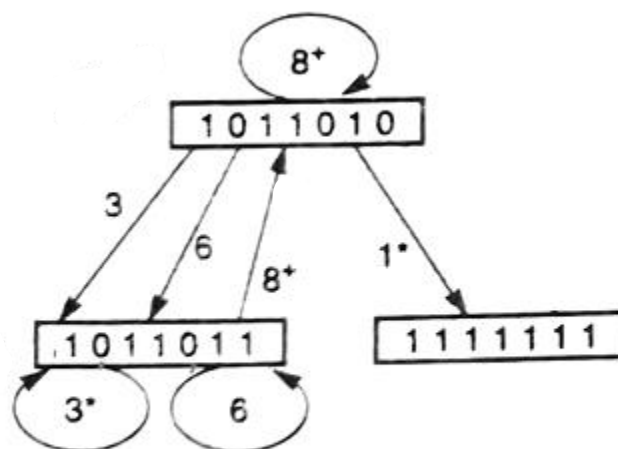
With latency : 8

101 1010

0000000

1011010 Next state

So the latency 8 returns to original ICV (1011010)



Intermediate State Diagram

Step 3: Calculating Next state with present State (PS)= 1111111

Permissible latencies (or transitions) = 8+

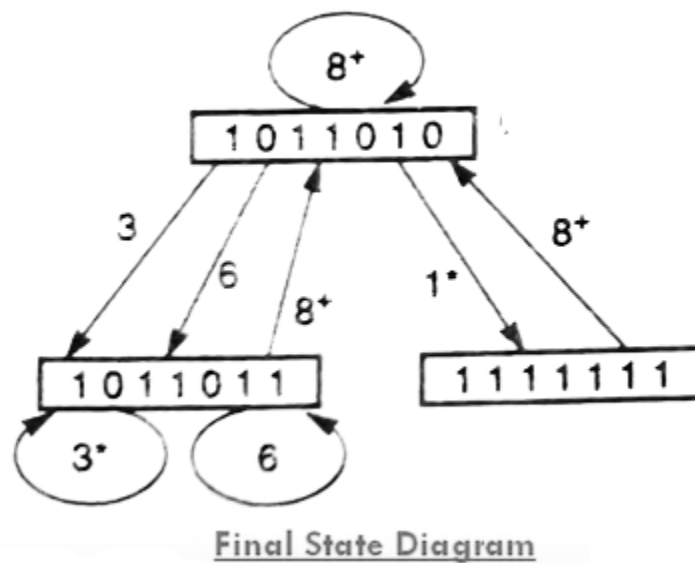
With latency 8+ :

1011010

0000000

1011010 Next State

So the latency 8 returns to original ICV (1011010)



Simple Cycle: A Simple Cycle is a latency cycle in which each state appears only once.

Greedy Cycle: A Greedy Cycle is a simple cycle whose edges are all made with minimum latencies from their respective starting states.

Their average latencies must be lower

Latency Cycles	Types of cycles	Average Latency
(3)	Simple cycle (Gray cycle)	3
(6)	Simple cycle	6
(8)	Simple cycle	8
(1,8)	Simple cycle (Gray cycle)	4.5
(3,8)	Simple cycle	5
(6,8)	Simple cycle	7
(3,6,3)		

MAL (Minimum Average Latency) is the minimum average latency obtained from the greedy cycle.

- The minimum-latency edges in the state diagrams are marked with asterisks.
- At least one of the greedy cycles will lead to the MAL

The Greedy Cycle leads to MAL = 3

• Vector Processor



Definition:

Vector processor is basically a central processing unit that has the ability to execute the complete vector input in a single instruction. More specifically we can say, it is a complete unit of hardware resources that executes a sequential set of similar data items in the memory using a single instruction.

- We know elements of the vector are ordered properly so as to have successive addressing format of the memory. This is the reason why we have mentioned that it implements the data sequentially.
- It holds a single control unit but has multiple execution units that perform the same operation on different data elements of the vector.

- Unlike scalar processors that operate on only a single pair of data, a vector processor operates on multiple pair of data. However, one can convert a scalar code into vector code. This conversion process is known as vectorization. So, we can say vector processing allows operation on multiple data elements by the help of single instruction.
- These instructions are said to be single instruction multiple data or vector instructions. The CPU used in recent time makes use of vector processing as it is advantageous than scalar processing.

- **Advantages of Vector Processor**

- Vector processor uses vector instructions by which code density of the instructions can be improved.
- The sequential arrangement of data helps to handle the data by the hardware in a better way.
- It offers a reduction in instruction bandwidth.